

COMPANION  
to

*ChatGPT Teaches TikZ*

*Appendix C: TikZ Error Messages and  
Debugging*

---

# Contents

|      |                                                        |    |
|------|--------------------------------------------------------|----|
| C.1  | What This Companion Adds . . . . .                     | 3  |
| C.2  | The First Rule: Read the First Error . . . . .         | 3  |
| C.3  | A Five-Step Diagnostic Sequence . . . . .              | 4  |
| C.4  | Error Messages at a Glance . . . . .                   | 5  |
| C.5  | Missing Semicolons and Broken Paths . . . . .          | 7  |
| C.6  | Unknown Commands, Keys, and Libraries . . . . .        | 8  |
| C.7  | Runaway Arguments and <code>pgfkeys</code> . . . . .   | 10 |
| C.8  | Nodes, Coordinates, and Names . . . . .                | 11 |
| C.9  | Styles and Pics . . . . .                              | 12 |
| C.10 | Errors Caused by the Document Infrastructure . . . . . | 14 |
| C.11 | Externalization and Build Errors . . . . .             | 15 |
| C.12 | When the Picture Compiles but Is Wrong . . . . .       | 15 |
| C.13 | A Compact Debugging Checklist . . . . .                | 16 |
| C.14 | Using ChatGPT as a Debugging Partner . . . . .         | 16 |

## C.1 What This Companion Adds

TikZ error messages can look more mysterious than the mistakes that produce them. A missing semicolon may be reported several lines later. An unknown option may appear as a `pgfkeys` error. A missing library may make a perfectly reasonable command look undefined. In a large picture, the line named in the log is often the place where TeX finally became unable to continue, not necessarily the place where the mistake was made.

This appendix is a practical reference for diagnosing such failures. It extends the systematic method of Chapter 29: read the first error, reduce the failing construction, and restore complexity one controlled change at a time.

### Principle

Treat an error message as evidence, not as a verdict about one line of source. The first question is not “Which line is guilty?” but “What was TeX trying to read when it lost the structure of the input?”

## C.2 The First Rule: Read the First Error

A single mistake can produce a cascade of later messages. Once TeX has lost track of a brace, path, key list, or environment, many later commands may be interpreted in the wrong context. Correcting the third or fourth message first can therefore waste time.

When compilation stops, begin with the first genuine error in the log. Record three things:

1. the exact error message;
2. the source line reported by TeX; and
3. the complete command or environment immediately before that line.

Then inspect the surrounding structure: braces, brackets, parentheses, environment boundaries, commas in option lists, and semicolons ending TikZ paths.

### Warning

The line number is a clue, not proof. A runaway argument or an unfinished path may begin many lines before the line on which TeX finally reports the failure.

## C.3 A Five-Step Diagnostic Sequence

The following sequence solves a large proportion of TikZ compilation problems.

1. **Reduce.** Replace the failing construction by the smallest version that should work.
2. **Compile.** Establish a known working point.
3. **Add one feature.** Restore one option, style, path operation, label, library-dependent feature, or reusable component.
4. **Compile again.** If the failure returns, the last change is strong evidence about the cause.
5. **Repeat.** Continue until the full construction has been restored.

For example, reduce a complicated connection to

Code

```
\draw (A) -- (B);
```

and only then restore its arrow tip, bend, label, decoration, and other options.

For a large block, another useful method is to comment out roughly half of it. If the remaining half compiles, the failure lies in

the removed half; otherwise it lies in the retained half. Repeating this process rapidly isolates a small region of source.

### Principle

One controlled change is more informative than several simultaneous changes. Always keep a known working version from which the next test can begin.

## C.4 Error Messages at a Glance

The exact wording can vary with the TeX engine and PGF/TikZ release, but the following forms are representative.

| Message or symptom                               | Usually means                                 | First check                                                                               |
|--------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------|
| Undefined control sequence                       | A command is unknown in the current document. | Spelling; required package or library; whether a defining style file was actually loaded. |
| Package pgfkeys Error: I do not know the key ... | TikZ does not recognize an option or key.     | Spelling; key path; required library; whether the key was defined before use.             |
| Package tikz Error: Giving up on this path       | TikZ could not finish parsing a path.         | Missing semicolon; malformed path operation; unmatched braces or brackets in the path.    |

Continued on next page

| Message or symptom                      | Usually means                                                          | First check                                                                              |
|-----------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| No shape named ... is known             | A referenced node or coordinate name is unavailable.                   | Exact name; definition order; scope; pic prefix or instance name.                        |
| Runaway argument?                       | TeX started reading an argument and did not find the expected end.     | Unmatched brace; paragraph break inside an argument; incomplete key or macro definition. |
| Paragraph ended before ... was complete | A paragraph break arrived while a macro was still reading an argument. | Blank line inside a fragile argument; missing closing brace; earlier structural error.   |
| Missing } inserted                      | TeX believes a group was left open.                                    | Braces around styles, nodes, labels, and nested TikZ definitions.                        |
| Extra }, or forgotten ...               | A closing group appeared without the matching opening structure.       | Nearby brace balance and recently edited groups.                                         |
| Missing number, treated as zero         | A numeric or dimensional value could not be parsed.                    | Units; commas; arithmetic expression; value supplied to a key.                           |
| File tikzlibrary... code.tex not found  | A requested library file cannot be located.                            | Library spelling; installed PGF/TikZ version; TeX installation.                          |

Continued on next page

| Message or symptom                         | Usually means                                                                                                          | First check                                                                        |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Picture compiles but an object is missing  | The command may be outside the visible region, clipped, transparent, behind another layer, or geometrically misplaced. | Coordinates; clipping; opacity; layer order; bounding box.                         |
| Picture compiles but the geometry is wrong | Syntax is valid but the construction does not express the intended relationships.                                      | Calculated coordinates, anchors, node dimensions, transformations, and path order. |

## C.5 Missing Semicolons and Broken Paths

Every TikZ path command must end with a semicolon. The mistake

Code

```
\draw (0,0) -- (3,0)
\draw (0,1) -- (3,1);
```

can cause TikZ to read the second command as if it were still part of the first path. The error may therefore be reported on the second line even though the omission is on the first.

When a long path fails, remove everything except one segment.

Code

```
\draw (A) -- (B);
```

Then restore operations one at a time.

Code

```
\draw[-{Stealth}] (A) -- (B);  
\draw[-{Stealth}] (A) to[bend left=20] (B);  
\draw[-{Stealth}] (A)  
  to[bend left=20]  
  node[above] {$f$}  
  (B);
```

At the first failing stage, inspect only the newly restored feature.

## C.6 Unknown Commands, Keys, and Libraries

An unknown command and an unknown key are different failures.

A message such as

Code

```
! Undefined control sequence.
```

means that TeX does not know the command name in the current context. Check spelling and determine what package or definition is supposed to provide it.

A message of the form



Code

```
Package pgfkeys Error:  
I do not know the key '/tikz/...'
```

means that the PGF key system could not find the requested key. The key may be misspelled, may belong to a library that was not loaded, or may be a custom key that has not yet been defined.

For example, relative positioning such as

Code

```
\node[right=of A] (B) {$B$};
```

requires the `positioning` library. Modern arrow tips such as `Stealth` require `arrows.meta`. Coordinate calculations of the form

Code

```
($ (A)!0.5!(B)$)
```

require `calc`.

If a library name already appears in the book's style file but the feature still seems unavailable, test the library directly in the main source file.

Code

```
\usetikzlibrary{decorations.pathmorphing}
```

If the small test then works, the library itself is present. Investigate which style file TeX is actually loading, whether an earlier error prevents the loading command from being reached, and whether the editor is compiling the intended main file.

## Principle

First prove that the smallest possible document can load and use the required library. Only then investigate the larger book infrastructure.

## C.7 Runaway Arguments and pgfkeys

A message such as

Code

```
Runaway argument?  
Paragraph ended before \pgfkeys... was complete.
```

is especially misleading because the closing brace reported by TeX may be perfectly correct. The real problem may occur earlier, while TeX is reading a nested key definition.

The first checks are:

1. Are all braces balanced inside `\tikzset`?
2. Are commas present between neighboring key definitions?
3. Is a blank line creating a paragraph break inside an argument that cannot accept one?
4. Did a preceding macro or environment fail to close correctly?
5. Does the same key definition work in a tiny test document?

Suppose a large definition fails. Replace it temporarily by one simple style.

Code

```
\tikzset{  
  vertex/.style={draw,circle}  
}
```

Then add the next style, compile, add the next feature, and compile again. This procedure distinguishes a genuine key-definition failure from a problem in the surrounding environment.

### Warning

Do not assume that the final brace named in a runaway-argument message is wrong. It may simply be the point at which TeX can no longer continue reading an argument that became malformed earlier.

## C.8 Nodes, Coordinates, and Names

A simple pattern is

### Code

```
\node (A) at (0,0) {$A$};  
\node (B) at (3,0) {$B$};  
\draw (A) -- (B);
```

If TikZ reports that no shape with a particular name is known, check the following.

1. Is the spelling and capitalization identical at definition and use?
2. Was the node defined before the path that refers to it?
3. Is the node inside a scope that has already ended?
4. Is the name created inside a pic, where instance prefixes or local names may change the external name?
5. Was a node renamed during editing while later references were left unchanged?

For pics, distinguish the name of the reusable pic from the name of a particular instance.

Code

```
\pic (A) at (0,0) {module};  
\pic (B) at (4,0) {module};
```

If the pic exposes coordinates such as `(-left)` and `(-right)`, the instance names allow references such as `(A-right)` and `(B-left)`.

## C.9 Styles and Pics

Reusable definitions reduce repetition, but they also create an additional layer to diagnose. When a style or pic fails, separate two questions:

1. Does the definition itself compile?
2. Does its use compile?

For a style, first define it without applying it.

Code

```
\tikzset{  
  highlightedvertex/.style={  
    draw,  
    very thick,  
    fill=yellow!30  
  }  
}
```

Then apply it to one object.

Code

```
\node[highlightedvertex] at (0,0) {$A$};
```

For a pic, first define a minimal reusable picture.

Code

```
\tikzset{
  pics/vertex/.style={
    code={
      \fill (0,0) circle (2pt);
    }
  }
}
```

Then place it once.

Code

```
\pic at (0,0) {vertex};
```

Only after that succeeds should additional styles, parameters, labels, and several instances be restored.

## Design

Use exact, consistent names for styles, pics, nodes, and coordinates. During debugging, short names without punctuation or accidental spacing make comparisons easier. Once the construction works, choose descriptive names that express the mathematical role of the object.

## C.10 Errors Caused by the Document Infrastructure

A TikZ example can be correct while the document that contains it is misconfigured. This is particularly important in a book that uses a class file, a project style file, and many chapter or companion files.

Typical infrastructure failures include:

- the intended style file is not loaded;
- TeX finds another file with the same name earlier in its search path;
- the editor compiles a chapter file instead of the main book file;
- an earlier error prevents TeX from reaching a later library-loading command;
- a required package is loaded only in one test document but not in the book; and
- stale auxiliary or externalized files obscure what is actually being compiled.

A decisive test is to copy the smallest failing example into a minimal document containing only its known requirements. If the minimal document works, the TikZ construction is probably sound and attention should turn to the larger document structure.

Code

```
\documentclass{article}
\usepackage{tikz}
\usetikzlibrary{positioning}

\begin{document}
\begin{tikzpicture}
  \node (A) {$A$};
  \node[right=of A] (B) {$B$};
  \draw (A) -- (B);
\end{tikzpicture}
\end{document}
```

## C.11 Externalization and Build Errors

Externalization introduces a second source of problems: the build process. A picture may compile normally but fail when externalization is enabled.

Check:

1. whether the **external** library is loaded;
2. whether externalization has been activated;
3. whether the TeX engine is allowed to launch the required auxiliary compiler process;
4. whether the cache directory exists and is writable;
5. whether a stale cached picture is being reused; and
6. whether shared style changes require a rebuild.

When the drawing itself is still under development, disable externalization and debug the TikZ source first. Re-enable the production mechanism only after the picture is stable.

## C.12 When the Picture Compiles but Is Wrong

Successful compilation proves only that TeX understood the syntax. It does not prove that the geometry is correct.

Temporary marks can expose calculated points.

Code

```
\coordinate (M) at ($(A)!0.5!(B)$);  
\fill (M) circle (1.5pt);
```

Temporary guides can reveal anchors and alignments. Removing styles can show whether a large node is changing spacing. Disabling clipping can show whether an object exists but lies outside the visible region. Likewise, temporarily removing opacity or background layers can expose an object hidden by drawing order.

### Principle

Do not repair an unexplained geometric error with a random shift. Find the coordinate, anchor, node size, transformation, clipping path, or layer order that created the discrepancy.

## C.13 A Compact Debugging Checklist

When a TikZ example fails, work through the following list in order.

1. Read the first error, not the last one.
2. Check the command immediately before the reported line.
3. Check semicolons ending paths.
4. Check matching braces, brackets, parentheses, and environments.
5. Check commas in option and key lists.
6. Check exact node, coordinate, style, and pic names.
7. Check that required libraries are loaded.
8. Reduce the failing construction to the smallest version that works.
9. Restore one feature at a time, compiling after each change.
10. If a minimal document works, investigate the class file, style file, main file, search path, and build configuration.
11. If the picture compiles but looks wrong, inspect geometry rather than syntax.
12. After the cause is understood, remove temporary diagnostic code.

## C.14 Using ChatGPT as a Debugging Partner

ChatGPT is most useful when it receives the smallest complete failing example together with the exact first error message. A productive prompt contains four pieces of information:



This TikZ example fails to compile. Identify the first actual cause, not merely the line where TeX stops. Explain the error briefly, then give the smallest correction. Do not redesign the picture. Here is the exact error message, followed by the complete minimal example.

For a large project, ask ChatGPT to propose one diagnostic experiment at a time. After each compile, report whether the test succeeded and keep the last working version. This turns debugging into a sequence of small proofs about what is and is not causing the failure.

### Principle

A good debugging session should end with an explanation, not merely a file that happens to compile. Knowing why the failure occurred prevents the same problem from returning in the next diagram.